

UNIVERSITY OF FORT HARE

Department of Computational Sciences

CSC 200 / CSC 300 Capstone Project

UFH Safety Platform

Software Documentation

A student safety, emergency and wellness platform.

This is an academic prototype: it is not connected to any live emergency service or GBV support unit, and all data in the repository is synthetic.

Prepared by:

Pair A – Foundation: Sinesipho Malgas, Nondumiso Mbuli

Pair B – Incidents: Brains Nkosi, Nkosinathi Mbewana

Pair C – Shell & Auth: Scott, Nondumiso Mkhonto

Pair D – Incidents UI: Mpilwenhle Jubane, Awethu Dyani

Repository: github.com/Mjubane5/ufh-safety-platform

September 2026

Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 This is a prototype	3
1.4 Companion documents	3
2. System Overview	4
2.1 Roles	4
2.2 High-level capabilities	4
3. System Architecture	5
3.1 Layers	5
3.2 Why no framework on the frontend	5
4. Security Model and Request Lifecycle	6
4.1 Authorization pattern	6
4.2 403 vs. 404 – an information-leak-aware design choice	6
4.3 Authentication	6
4.4 Two-step student login	7
5. Data Design	8
5.1 Core entities	9
5.2 Soft deletion	9
6. Module Design	10
6.1 Backend packages	10
6.2 Frontend structure	10
7. Interface Design	11
7.1 Conventions (apply to every endpoint)	11
7.2 Endpoint groups	11
8. Incident Lifecycle	12
9. Software Engineering Practices	13
10. Testing Strategy	14
10.1 Approach	14
10.2 What is covered, by package	14
10.3 Known gap	14
11. Known Limitations and Deferred Work	15
12. Deployment	16
12.1 Hosting	16

12.2 Configuration	16
12.3 Local development	16
13. References	17

1. Introduction

1.1 Purpose

This document is the technical software documentation for the UFH Safety Platform: a description of what the system is built from, how its parts fit together, and the engineering decisions behind it. It is written for an examining panel, a future maintainer, or a teammate who needs to understand a part of the system they did not build themselves.

1.2 Scope

The UFH Safety Platform is a campus safety, emergency and wellness system for the University of Fort Hare, built as a CSC 200 / CSC 300 collaborative capstone project. It lets students report incidents, trigger an SOS, use a live safety map with a Safe Walk feature, book and message the Student Counselling Unit and Campus Health Centre separately, and file confidential GBV reports; campus control dispatches responders and logs patrols; and a GBV officer manages confidential cases through a channel that never exposes reporter identity.

1.3 This is a prototype

Not connected to any real emergency service, campus authority, or GBV support unit. Every account, incident and report in this system is synthetic. Nothing typed into it reaches anyone outside the project. This is stated here, in the README, and in the project status report, consistently, because the risk of a reader mistaking a demo for a real safety channel is the single most serious thing this document could get wrong.

1.4 Companion documents

This document focuses on architecture, design and engineering practice. It deliberately does not duplicate:

- **docs/api-contract.md** – the field-by-field source of truth for every request and response.
- **docs/PROJECT-STATUS-REPORT.md** – scope changes, timeline, bug log and the individual contribution matrix.
- **backend/README-BACKEND.md** – local setup steps, environment variables, and the SendGrid deliverability writeup.
- **docs/team-working agreement_1.md** – pairing, ownership and the rules the team agreed to follow.

2. System Overview

2.1 Roles

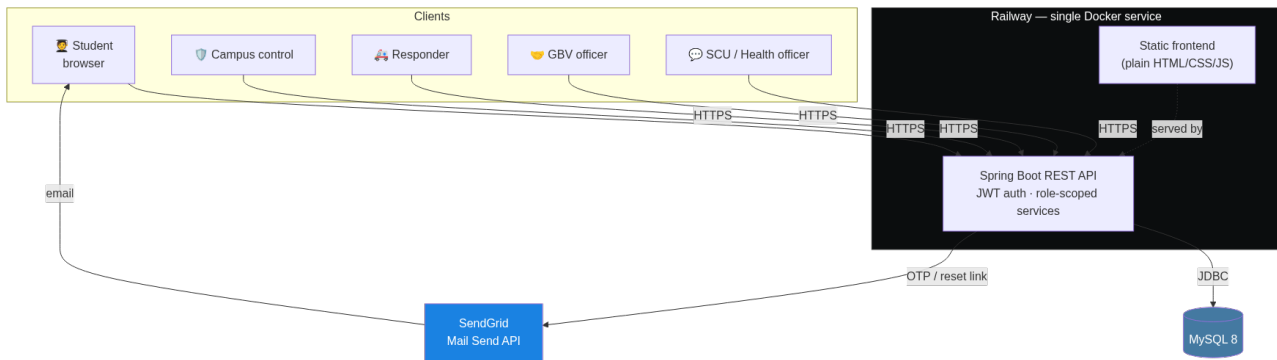
Five roles share one system, each seeing only what their role permits. Role checks are enforced server-side on every request – the frontend hiding a button is a convenience, never the security boundary.

Role	Can do
student	SOS, incident reports, Safe Walk, wellness bookings/chat, GBV reporting, health profile, contact campus control
campus_control	Dispatch queue, responder assignment, patrol logging, active Safe Walk monitoring, student contact queue
responder	Assigned-incident view, status updates (en_route / on_scene / resolved)
gbv_officer	Confidential case queue and status, chat scoped to a report's reference code, never to a student account
scu_officer / health_officer	Separate booking queues and message threads for counselling vs. physical health

2.2 High-level capabilities

- One-tap SOS and a 9-type incident report form, usable with or without a device location.
- A live Leaflet + OpenStreetMap safety map: recent patrols, risk hotspots, and a safer-route calculation for Safe Walk.
- Safe Walk: live location sharing on a chosen route until arrival, with a simulated-walk demo mode for presentations.
- Two-step student login: password, then a 6-digit emailed code, before a session token is issued.
- Confidential GBV reporting, anonymous or not, with a reference-code status lookup and structurally anonymous chat.
- Separate wellness (SCU) and physical-health (Campus Health Centre) booking and messaging channels.

3. System Architecture



One Spring Boot service serves the REST API and the built frontend from the same origin. This was a deliberate pivot from an earlier three-service sketch: same-origin means there is no cross-origin request to secure or debug in production, and one deployment updates both halves together.

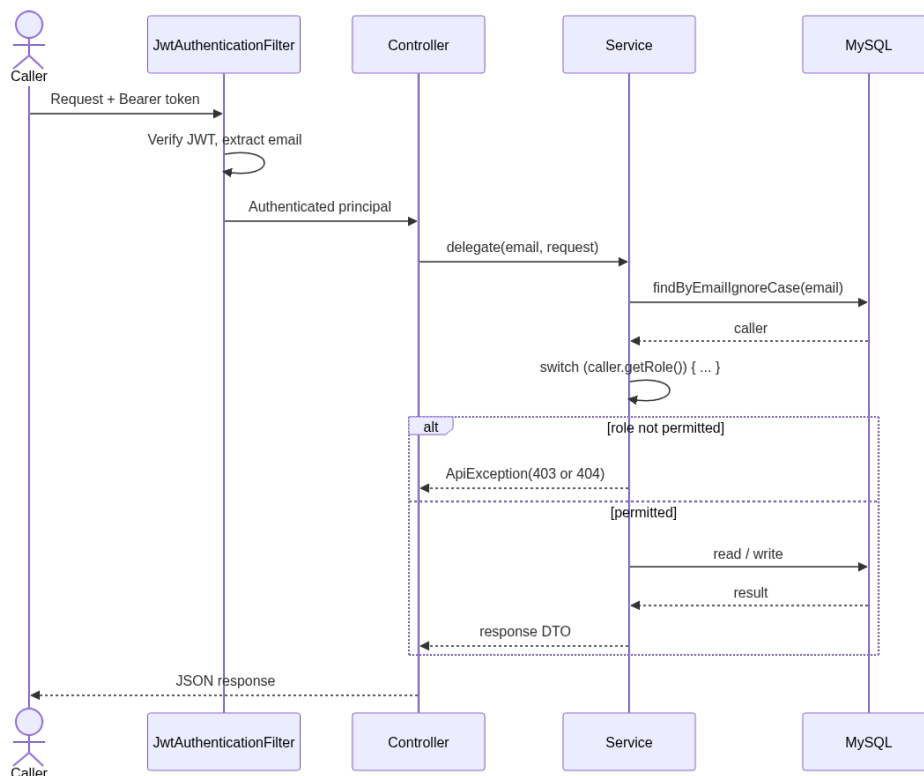
3.1 Layers

Layer	Responsibility
Frontend	Plain HTML/CSS/JavaScript, mobile-first. Every network call goes through frontend/js/api.js – no fetch() anywhere else in the codebase.
Controller	Spring @RestController classes. Parse the request, delegate to a service, return a DTO. No business logic.
Service	Where every rule lives: role checks, state transitions, validation, orchestration. Framework-agnostic – no HttpServletRequest below this layer (one documented exception, see 4.2).
Repository	Spring Data JPA interfaces. Thin, declarative, no hand-written SQL except where a derived query cannot express the need.
MySQL	Single relational store. No BLOBs, no document store – evidence and attachments are file paths, per the team's own decision log.

3.2 Why no framework on the frontend

Plain HTML/CSS/JavaScript was chosen so every page works against fake in-memory data before the backend exists (frontend/js/config.js's MOCK flag), and against the real backend afterward, with zero page code changing – the mock responses are built to match the API contract's shapes field for field.

4. Security Model and Request Lifecycle



Every protected request follows this same path, in every service in the codebase.

4.1 Authorization pattern

There is no `@PreAuthorize` and no centralized authorization framework in this codebase. Every service resolves the caller by email (`users.findByEmailIgnoreCase`), then runs a plain `switch (caller.getRole())`, throwing an `ApiException(status, code, message, field)` for anything not permitted. This was a deliberate choice for a student team: the rule for any endpoint is a few lines in one method, not a separate annotation layer to learn and keep in sync with it.

4.2 403 vs. 404 – an information-leak-aware design choice

A caller who is not allowed to know a resource exists at all gets `404`, not `403`. Returning `403` would itself confirm the resource is there – for a GBV report in particular, that distinction is not a nicety, it is the difference between "I cannot say" and "yes, but not for you." See `IncidentService.requireCanView` and `GbvReportService.requireReport` for the concrete implementation.

4.3 Authentication

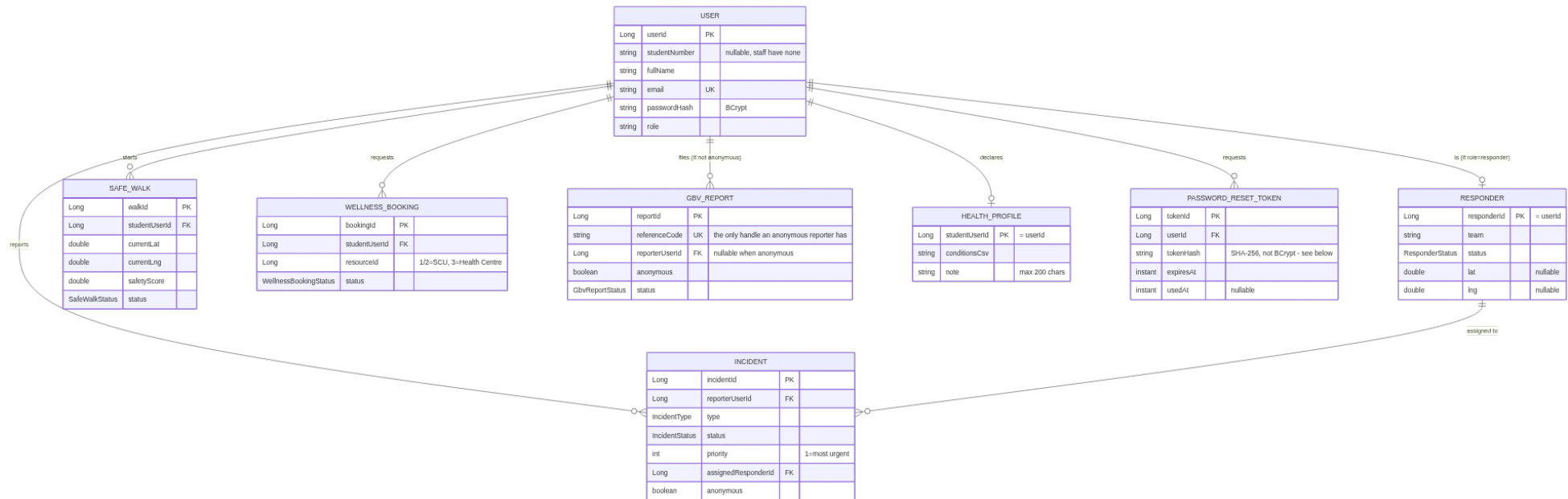
- JWT bearer tokens (`Authorization: Bearer <token>`), fixed 8-hour lifetime, no refresh token – simpler, and adequate for a prototype.
- Passwords and login codes are hashed with **BCrypt**: slow by design, which resists brute-forcing a low-entropy secret a human chose or a 6-digit code.
- Password-reset tokens use **SHA-256** instead: a 256-bit `SecureRandom` value does not need a slow hash, and needs to be a fast, direct, queryable lookup key. The reasoning is written down as a class comment on `PasswordResetToken`, not left implicit.

- ``JwtAuthenticationFilter`` verifies the token and resolves the caller's email once per request, ahead of every controller.

4.4 Two-step student login

Student sign-in is two steps: password, then a 6-digit code emailed to that account, entered before a session token is issued – so a student is notified, and must act, every time their account signs in, not just warned after the fact. Staff logins are unaffected; they call ``POST /api/auth/login`` directly. As of this document, ``login.html`` also accepts a bare student number in place of an email: ``AuthService.verifyCredentials`` looks up by the digits-only ``studentNumber`` field when the typed value has that shape, and by email otherwise – both paths return the same generic 401 for an unknown identifier or a wrong password, so neither can be used to enumerate valid accounts.

5. Data Design



Every foreign key in this schema is a plain `Long`/`String` column, resolved by hand in the service layer – there are no JPA `@ManyToOne` relationships anywhere in the codebase. This was a deliberate, consistent choice: it keeps every visibility rule (who may read which row) in one auditable method instead of letting it be implied by an object graph a reader would have to trace through.

5.1 Core entities

Entity	Purpose
User	One row per account. studentNumber is nullable (staff have none); role is set server-side, never from the request body, so a caller cannot register themselves as an admin.
Incident	A report: type, status, priority (1 = most urgent), optional location, optional anonymity, optional assigned responder.
Responder	Duty status and last-known location for a user with role=responder.
SafeWalk	A live-tracked walking session: origin, destination, current position, a computed safety score, and status.
Hotspot / Patrol	Risk-area and patrol-history data feeding the safety map and the safer-route calculation.
GbvReport / GbvMessage	Confidential reports keyed by a non-guessable reference code, and a chat scoped to that code rather than a user.
WellnessBooking / WellnessMessage	SCU bookings and messages, kept in a separate table and queue from health.
HealthMessage / HealthProfile	Campus Health Centre messages, and a student's own declared health conditions for responders.
PasswordResetToken	A single-use, time-limited, SHA-256-hashed reset token.
IncidentSignal	Live, appendable updates on an incident (e.g. a location ping) without overwriting the incident row itself.

5.2 Soft deletion

Incidents are never hard-deleted. A false alarm sets `status = cancelled`, it does not remove the row – there is always a record to review, and no endpoint in the contract deletes an incident outright.

6. Module Design

6.1 Backend packages

``backend/src/main/java/za/ac/ufh/safety/``:

Package	Responsibility
auth	Registration, login (email or student number), the two-step emailed code, JWT issuance, forgot/reset password wiring.
user	The User entity and its repository. Deliberately small – everything role-specific lives in the package that needs it.
incidents	Report -> triage -> assign -> resolve/cancel lifecycle, priority calculation, live incident signals.
responders	Duty status, location, and nearest-responder selection for dispatch.
safetywalk	Hotspots, patrols, safer-route calculation, and live Safe Walk sessions.
wellness	SCU booking and messaging, kept separate from health.
health	Campus Health Centre messaging (bookings are shared with wellness's resource model).
healthprofile	A student's own declared health conditions, shown to a responding officer.
campuscontrol	The general student-to-campus-control contact channel, outside the incident flow.
gbv	Confidential reporting, status lookup, and a chat structurally scoped to a report's reference code.
passwordreset	Forgot/reset password: SHA-256 token issuance and single-use redemption.
common	The shared ApiException / ErrorResponse / GlobalExceptionHandler – one error shape for the whole API.
config	Security configuration, the JWT filter, and the demo-data seeder used in development and on Railway.

6.2 Frontend structure

``frontend/``: one HTML page per screen, one same-named JavaScript module behind it, and three shared modules every page can import:

- **api.js** – the only file allowed to call ``fetch()``. Every other module imports from here.
- **config.js** – the single place a base URL, the MOCK flag, and the prototype-notice flag are set.
- **tracking.js** – shared geolocation/distance helpers used by both Safe Walk and live incident tracking.

Representative pages: ``dashboard.html`` (student home), ``report.html`` (incident form), ``map.html`` (Safe Walk), ``control-dashboard.html`` (campus control), ``responder-dashboard.html``, ``gbv.html`` / ``gbv-officer.html``, ``wellness.html``, ``health-profile.html``, and the shared ``login.html`` / ``register.html`` / ``staff-login.html`` / ``forgot-password.html`` / ``reset-password.html`` auth flow.

7. Interface Design

`docs/api-contract.md` is the single source of truth for every request and response shape – field names, types, and error cases. This section is a summary by category, not a replacement for it.

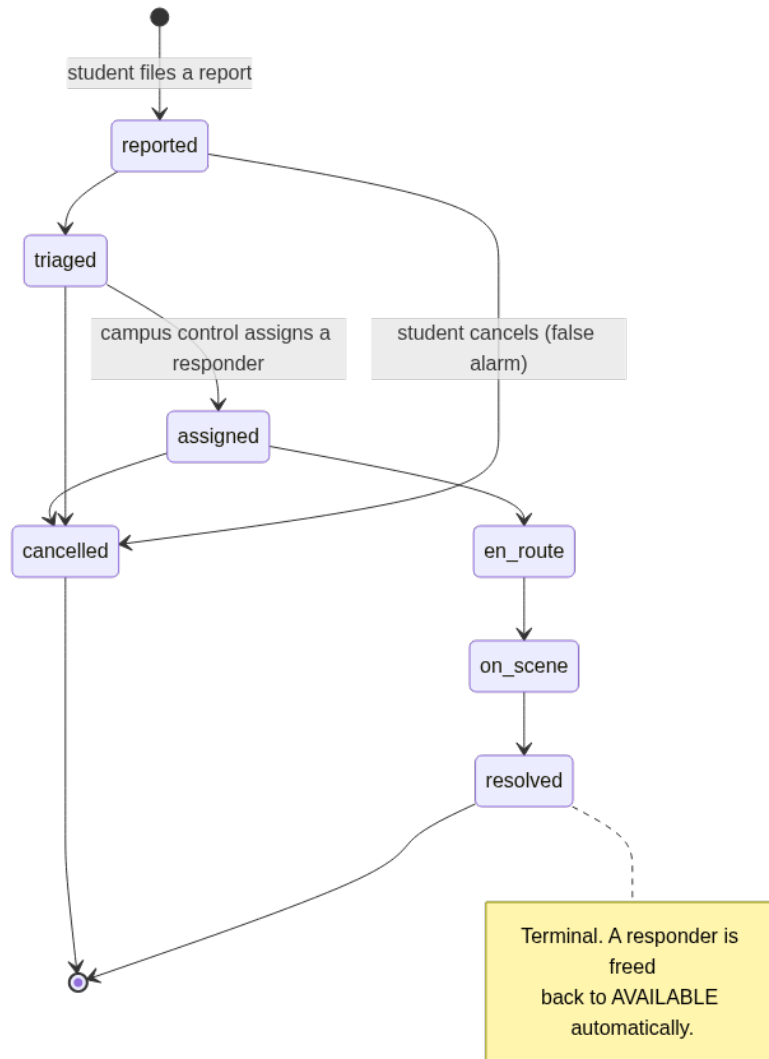
7.1 Conventions (apply to every endpoint)

- `camelCase` field names, always – never `snake\_case`.
- Timestamps: ISO 8601 UTC with a `Z` suffix.
- Coordinates: `latitude` and `longitude`, spelled in full.
- Empty values: `null`, never `""` or `-1`.
- Every non-2xx response: `{ "error": ..., "message": ..., "field": null }`.
- Auth: `Authorization: Bearer <token>` on every protected endpoint.

7.2 Endpoint groups

Group	Endpoints (representative)
Auth	POST /auth/register, /auth/login, /auth/login/request-code, /resend-code, /verify-code, /auth/forgot-password, /reset-password, GET /auth/me
Incidents	POST /incidents, GET /incidents, GET /incidents/{id}, PATCH /incidents/{id}/status, POST /incidents/{id}/cancel, POST /incidents/{id}/assign, GET/POST /incidents/{id}/signals
Responders	GET /responders, PATCH /responders/{id}/status
Safety / Safe Walk	GET /hotspots, GET /patrols/recent, POST /patrols, POST /routes/safe, POST /safewalks, PATCH /safewalks/{id}/location, POST /safewalks/{id}/arrived, /cancel, GET /safewalks/active
Wellness / Health	GET /wellness/resources, POST /wellness/bookings, GET/POST wellness and health messages, PATCH booking status
Campus control contact	POST /campus-control/messages, GET /campus-control/messages, GET /campus-control/queue
GBV	POST /gbv/reports, GET /gbv/reports/{code}/status, GET /gbv/reports (officer), PATCH /gbv/reports/{code}/status, GET/POST /gbv/reports/{code}/messages
Health profile	GET/PUT /students/me/health

8. Incident Lifecycle



A fixed state machine, enforced server-side by `StatusTransitions.isLegal(current, target)`, which rejects any jump that skips a step with `'409 Conflict'`. A terminal state (`'resolved'` or `'cancelled'`) can never move again. When an incident reaches `'resolved'`, the assigned responder is freed back to `'AVAILABLE'` automatically – this was an explicitly-flagged gap in an early scaffold, now covered by `'IncidentAssignmentServiceTest'`.

9. Software Engineering Practices

Concrete choices, each traceable to a specific file or test – not abstract claims about "best practice."

Practice	Where it shows up
Contract-first API design	docs/api-contract.md agreed before implementation for every endpoint; if code and contract disagree, the contract wins.
Layered architecture	Controller -> Service -> Repository throughout. Controllers never touch a repository directly.
Information-leak-aware errors	404 (not 403) for a resource a caller is not allowed to know exists – see section 4.2.
Anonymity enforced structurally	GbvReportItem and the GBV chat queue response have no field for reporter identity at all – not a null one. A reflection test asserts the record's components directly, so a future change cannot leak it by accident.
Threat-matched hashing	BCrypt for low-entropy secrets (passwords, login codes); SHA-256 for a 256-bit random reset token – see section 4.3.
No-enumeration guarantees, tested	Forgot-password returns an identical response for a registered and an unregistered email, asserted directly in PasswordResetServiceTest, not just documented.
Rate limiting where it matters most	The public GBV status-lookup endpoint is rate-limited per caller IP – brute-forcing a reference code by trying many, fast, is the realistic attack.
Test strategy matched to risk	189 pure unit tests (JUnit 5 + Mockito + AssertJ, no database) targeting the actual security boundaries: every role/endpoint pair, every 403-vs-404 decision, every idempotency case.
Mobile-first, verified in-browser	Every page built for a narrow viewport first; layout bugs this project shipped were caught by testing real breakpoints in a browser, not assumed from the CSS.
Honest documentation of limitations	Known gaps are written where the next person will find them – see section 11 – not hidden.

10. Testing Strategy

10.1 Approach

The backend test suite is **189 pure unit tests** – JUnit 5, Mockito and AssertJ, with every repository mocked, so the whole suite runs in well under a minute and needs no database. This was a deliberate trade-off for a student team on a deadline: fast, deterministic tests that run on every machine without setup, at the cost of not exercising the real JPA/MySQL mapping layer end to end (that gap is covered by manual testing against a real local database before each merge, and the `.http` files kept under `docs/` as evidence of it).

10.2 What is covered, by package

Package	Tests	What they assert
auth	20	Every login path (email, student number, staff), the two-step flow, no-enumeration on wrong password vs. unknown account, hashing, JWT signing.
incidents	44	Status transitions, priority calculation, assignment, visibility per role, signals.
safetywalk	32	Safe-route avoidance, Safe Walk lifecycle, hotspot/patrol data shaping.
gbv	22	Confidential queue visibility, status transitions, anonymity structurally enforced.
wellness / health / healthprofile / campuscontrol	29	Booking status rules, message-queue visibility, role scoping.
passwordreset	8	Token issuance, single use, expiry, no-enumeration.
responders / config / common	20	Nearest-responder selection, demo-seeder counts, the shared error handler.

10.3 Known gap

`request-code` and `resend-code` (the two-step login endpoints) have no rate limit per account yet, unlike the GBV status lookup. A caller who already knows a valid email/password and student number could currently spam that student's inbox by calling either endpoint repeatedly. This is recorded in `docs/api-contract.md` as a known gap, worth the same per-caller sliding-window treatment the GBV endpoint already has before this goes anywhere near real students.

11. Known Limitations and Deferred Work

Recorded here rather than left for someone to discover – each of these is a deliberate, disclosed trade-off, not an oversight found after the fact.

- **SendGrid Domain Authentication is not configured.** Email sends successfully but can land in spam, or in a Microsoft 365 tenant's quarantine, without full SPF/DKIM alignment on a domain the team actually controls the DNS for. Documented in detail, including the exact failure mode observed, in backend/README-BACKEND.md.
- **No rate limit on the two-step login's request-code/resend-code.** See section 10.3.
- **GBV evidence upload is out of scope for this pass.** Multipart upload and EXIF-stripping are separate security work, not assumed here.
- **No refresh token.** Sessions are a fixed 8-hour JWT – simpler, and an agreed, adequate trade-off for a prototype (see the team's decision log).
- **No C++ or Python module exists in this repository**, despite early planning documents listing a standalone C++ algorithms module and an offline Python analysis step. The corresponding work – nearest-responder selection, safe-route avoidance, hotspot risk scoring – was implemented in Java instead (`NearestResponderSelector.java`, `SafeRouteService.java`, `HotspotService.java`), for team-velocity and single-stack reasons under the project's time constraints. This is disclosed here rather than left for a reader to notice against the original plan.
- **No CI pipeline.** There is no `.github/workflows` directory in this repository; the test suite is run locally before every merge, not automatically on push.

12. Deployment

12.1 Hosting

A single Docker service on Railway. The Spring Boot backend serves the built frontend from the same origin, so there is no cross-origin request to secure in production – a deliberate pivot away from an earlier multi-service sketch, made explicitly to reduce operational surface for a student team with no dedicated DevOps capacity.

12.2 Configuration

Configuration is entirely environment-variable driven – nothing is hardcoded in source:

Variable	Purpose
JWT_SECRET	Signing key for session tokens. No default – the backend will not start without one.
DB_PASSWORD (etc.)	MySQL connection credentials.
MAIL_API_KEY / MAIL_FROM	SendGrid credentials. If unset, the backend still starts and the OTP/reset link is written to the server log instead of emailed – a dev-only fallback, never sent over HTTP to the browser either way.
APP_SEED_ENABLED	Whether the demo-data seeder provisions synthetic accounts on startup.

12.3 Local development

``backend/README-BACKEND.md`` has the full setup. In short: ``.\mvnw spring-boot:run`` for the backend (needs ``JWT_SECRET`` set; MySQL locally or a portable instance), and ``python -m http.server 5500`` for the frontend, which runs entirely on mock data by default with no backend required.

13. References

- **docs/api-contract.md** – the API contract.
- **docs/PROJECT-STATUS-REPORT.md** – scope, timeline, bug log, contribution matrix.
- **backend/README-BACKEND.md** – local backend setup and the SendGrid writeup.
- **docs/team-working agreement_1.md** – team pairing and rules.
- **docs/development-challenges.md** – challenges encountered and how they were resolved.
- **README.md** – project overview, architecture diagram, getting started.